

● CLOUD COST · TEST GUIDE

Testing the AWS China Relay in a Sandbox

For teams testing the AWS China relay end to end in an ordinary AWS account, before connecting real China data.

STAND-IN CHINA

us-west-1

RELAY AND DESTINATION

us-east-1

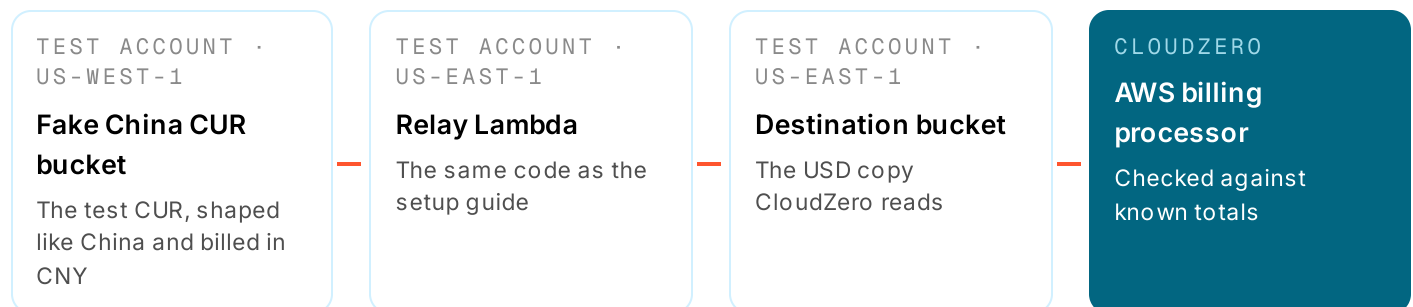
TEST DATA

Your own CUR export

This guide runs the AWS China relay end to end in an ordinary AWS account, without an AWS China account. A bucket in `us-west-1` stands in for the China account's CUR bucket and holds a fake China CUR, built from one of your own commercial CUR exports. Everything else, including the relay, the destination bucket and the CloudZero connection, is set up exactly as in the setup guide.

Because the fake CUR is built at a known rate, the relay's output can be checked against your original export to the cent.

How It Works



REAL SETUP	THIS TEST
China CUR bucket in <code>cn-northwest-1</code> , in the <code>aws-cn</code> partition	A bucket in <code>us-west-1</code> in your test account
Real China CUR, billed in CNY	A fake China CUR made by <code>make_fake_china_cur.py</code> from your own export
IAM user in the China account, with an access key	IAM user in your test account, with an access key. The relay reads with a key either way, so the same code path is tested.
Destination bucket and relay in your commercial account	Destination bucket and relay in <code>us-east-1</code> in your test account
IAM policy ARNs use <code>arn:aws-cn:</code> for the China bucket	All ARNs use <code>arn:aws:</code>

Before You Start

● KEEP YOUR TEST DATA PRIVATE

The fake CUR contains your real account data: resource IDs, resource names and tags.

Only upload it to accounts and CloudZero organizations you control. Use a test CloudZero organization, or one you're comfortable adding test spend to.

Prerequisites

- An AWS account where you can create IAM users and roles, S3 buckets, Secrets Manager secrets and Lambda functions
- That account connected to a CloudZero organization, so you can create an S3 Bucket connection in Step 7
- One billing period of a Legacy CUR export in CSV format, ideally with the `Manifest.json` AWS writes alongside it
- `relay.py` and `make_fake_china_cur.py` from this repository
- The AWS CLI, signed in to that account, and Python 3.9 or later

● STEP 01

Build and Upload the Fake China CUR

1. Build the fake CUR from your export. Leave out `--add-china-extras` for this test, so every value converts back to your original exactly:

```
python3 make_fake_china_cur.py ./YOUR_EXPORT_FOLDER/ --out ./fake-cn \
--prefix cur --report china-hourly --bucket YOUR_SOURCE_BUCKET --remap-accounts
```

REPLACE BEFORE PASTING

- `YOUR_EXPORT_FOLDER`: the folder holding your commercial CUR export for one billing period
- `YOUR_SOURCE_BUCKET`: the stand-in China bucket you create in us-west-1 in Step 1

2. Note your export's total unblended cost. It's the number the relay and CloudZero should reproduce:

```
python3 -c "import csv,gzip,glob,sys; t=0
for f in glob.glob('YOUR_EXPORT_FOLDER/**/*.csv*', recursive=True):
    o=gzip.open(f,'rt',newline='') if f.endswith('.gz') else open(f,newline='')
    t+=sum(float(r['lineItem/UnblendedCost'] or 0) for r in csv.DictReader(o))
print(round(t,2))"
```

REPLACE BEFORE PASTING

- `YOUR_EXPORT_FOLDER`: the folder holding your commercial CUR export for one billing period

3. In the S3 console, switch to **US West (N. California) us-west-1** and create a general purpose bucket with the defaults: ACLs disabled, Block all public access on and SSE-S3 encryption. Then upload the fake CUR:

```
aws s3 sync ./fake-cn/ s3://YOUR_SOURCE_BUCKET / --region us-west-1
```

REPLACE BEFORE PASTING

- `YOUR_SOURCE_BUCKET`: the stand-in China bucket you create in us-west-1 in Step 1

● EXPECTED RESULT

The bucket contains `cur/china-hourly/YYYYMMDD-YYYYMMDD/china-hourly-Manifest.json` and one assembly folder holding the report file and a second manifest. The generator prints the row count and billing period. Note both for Step 6.

● STEP 02

Create the Reader User and Store Its Key

In the IAM console, create a user named `test-china-cur-reader` with no console access and attach this inline policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::YOUR_SOURCE_BUCKET",
      "Condition": { "StringLike": { "s3:prefix": ["cur/*"] } }
    },
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::YOUR_SOURCE_BUCKET/cur/*"
    }
  ]
}
```

REPLACE BEFORE PASTING

- `YOUR_SOURCE_BUCKET`: the stand-in China bucket you create in us-west-1 in Step 1

Create an access key for the user. Then, in Secrets Manager in `us-east-1`, store it as a secret named `aws-china-cur-reader`, using **Other type of secret → Plaintext**:

```
{ "access_key_id": "YOUR_ACCESS_KEY_ID", "secret_access_key": "YOUR_SECRET_ACCESS_KEY" }
```

REPLACE BEFORE PASTING

- `YOUR_ACCESS_KEY_ID`: the access key ID for the reader user
- `YOUR_SECRET_ACCESS_KEY`: the secret access key for the reader user

● WATCH FOR STRAY SPACES

A space after a bucket name, easy to pick up when copying, makes the ARN match nothing. The result is an AccessDenied error that looks like a missing permission.

● STEP 03

Create the Destination Bucket

In the S3 console, switch to **US East (N. Virginia) us-east-1** and create a second bucket with the same defaults: ACLs disabled, Block all public access on, versioning off and SSE-S3 encryption. No bucket policy is needed. The relay creates its folders on the first run. S3 bucket names are unique across all AWS accounts, so pick a test-only name, for example one that includes your account ID.

● STEP 04

Upload the Test Exchange Rate

Save this as `cny_usd_monthly.json`, using your billing month as the key, and upload it. `0.13986014` is $1 \div 7.15$, which reverses the rate the generator used.

```
{ "YYYY-MM": "0.13986014" }
```

```
aws s3 cp cny_usd_monthly.json s3://YOUR_DEST_BUCKET/fx/cny_usd_monthly.json
```

REPLACE BEFORE PASTING

- `YOUR_DEST_BUCKET`: the destination bucket you created in us-east-1 in Step 3

● STEP 05

Deploy the Relay

Follow Step 5 of the setup guide, all in `us-east-1`, with the differences below. Skip **4. Schedule it**, because you'll run each test by hand.

1. Execution role

Create the `cloudzero-china-cur-relay` role with `AWSLambdaBasicExecutionRole` and this inline policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:us-east-1:YOUR_ACCOUNT_ID:secret:aws-china-cur-reader-*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::YOUR_DEST_BUCKET"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": [
        "arn:aws:s3:::YOUR_DEST_BUCKET/china-cur/*",
        "arn:aws:s3:::YOUR_DEST_BUCKET/cur-relay-audit/*",
        "arn:aws:s3:::YOUR_DEST_BUCKET/fx/*"
      ]
    }
  ]
}
```

REPLACE BEFORE PASTING

- `YOUR_ACCOUNT_ID`: your test account ID
- `YOUR_DEST_BUCKET`: the destination bucket you created in us-east-1 in Step 3

2. Function and settings

Create `cloudzero-china-cur-relay` on Python 3.12. To use the role, open **Additional settings**, turn on **Custom execution role**, choose `cloudzero-china-cur-relay` and select **Save** before **Create function**. Then paste the code from [Appendix A](#) into `lambda_function.py` and select **Deploy**. Set memory to 2048 MB, ephemeral storage to 10240 MB and the timeout to 15 minutes.

3. Environment variables

VARIABLE	TEST VALUE	WHY
CN_REGION	us-west-1	The stand-in China bucket's Region
CN_CUR_BUCKET	YOUR_SOURCE_BUCKET	The bucket from Step 1
CN_CUR_PREFIX	cur	Set by --prefix in Step 1
CUR_REPORT	china-hourly	Set by --report in Step 1
CN_SECRET_ID	aws-china-cur-reader	The secret from Step 2
DEST_BUCKET	YOUR_DEST_BUCKET	The bucket from Step 3
DEST_PREFIX	china-cur	Same as the setup guide
FX_RATES_KEY	fx/cny_usd_monthly.json	The file from Step 4

● STEP 06

Run and Check the Relay

On the function's **Test** tab, run an event naming your billing month:

```
{"periods": ["YYYY-MM"]}
```

● DON'T TEST WITH AN EMPTY EVENT

An empty `{}` event processes the current month, plus the previous month only during the first 10 days. Unless your export is from one of those months, name the month explicitly.

Check each of these:

CHECK	EXPECTED
Log output	One line for the month with your row count, and USD equal to your Step 1 total
Audit record in cur-relay-audit/	rows matches the generator's count, unblended_usd matches your Step 1 total, and converted_columns lists your report's money columns
Report in china-cur/china-hourly/	A top-level manifest plus one assembly folder with the converted file and its manifest. The manifest's bucket is your destination bucket.
Run the same event again	already up to date , with nothing new written

To spot-check the converted file, download it and confirm every row's currency is USD:

```
aws s3 cp s3://YOUR_DEST_BUCKET/china-cur/china-hourly/ ./out --recursive
python3 -c "import csv,gzip,glob,collections; c=collections.Counter()
for f in glob.glob('out/**/*.csv.gz', recursive=True): c.update(r['lineItem/CurrencyCode'] for r in
csv.DictReader(gzip.open(f, 'rt', newline='')))
print(c)"
```

REPLACE BEFORE PASTING

- `YOUR_DEST_BUCKET`: the destination bucket you created in us-east-1 in Step 3

● EXPECTED RESULT

Only `USD`, with a count equal to your row count. Use a CSV parser rather than `cut`, because the China entity names contain commas.

● STEP 07

Connect to CloudZero and Request AWS Processing

Follow Step 7 of the setup guide with these values:

Connection Name	aws-china-cur-test
Cloud Provider	AWS China
Bucket Name	YOUR_DEST_BUCKET
Bucket Path	china-cur/china-hourly

Attach the generated policy to the IAM role CloudZero uses for this account. Then ask your CloudZero contact to switch the connection to AWS billing processing.

● AN ERROR UNTIL THE SWITCH IS EXPECTED

Until the switch, CloudZero reads the path as a custom cost source, can't find the file it expects and shows an error on the connection.

● STEP 08

Validate in CloudZero

After the switch, allow up to 24 hours for processing. In Explorer, select your billing month, group by **Cloud Provider** and use **Billed Cost**.

CHECK	EXPECTED
Total for <code>Ningxia</code> plus <code>Beijing</code>	Your Step 1 total
Region values	<code>cn-northwest-1</code> and <code>cn-north-1</code>
Usage types	<code>CNW1-</code> and <code>CNN1-</code> prefixes

● STEP 09

Test a Month-End Restatement

This checks that CloudZero replaces a month when the relay publishes a new version, which the month-end close depends on.

1. Change the rate in `cny_usd_monthly.json` to `"0.1398"` and upload it again.
2. Run the same test event. The log shows a new version, at your Step 1 total $\times 0.99957$.
3. After CloudZero reprocesses, the month's Billed Cost should show that new total.

● WHAT A FAILURE LOOKS LIKE

If the month shows about twice your total, CloudZero added the new version on top of the old one instead of replacing it. Raise it with your CloudZero contact before relying on month-end restatements.

● STEP 10

Clean Up

When testing is done, remove what you created, in this order:

1. Delete the `aws-china-cur-test` connection in CloudZero.
2. Delete the Lambda function and its role.
3. Delete the reader user's access key, then the user and the `aws-china-cur-reader` secret.
4. Empty and delete both buckets.

● REFERENCE

Troubleshooting

1. **"No China CUR yet, skipping"**. The relay can't find the source manifest. Check `CN_REGION`, `CN_CUR_BUCKET` and `CN_CUR_PREFIX`, and that Step 1's upload kept the `cur/` folder.
2. **"No CNY to USD rate for YYYY-MM"**. The rates file is missing, or doesn't have that month as a key.
3. **AccessDenied reading the source**. The reader user's policy must use `arn:aws:` and your source bucket name, and the secret must hold the user's current key.
4. **AccessDenied on the destination**. Check the execution role's inline policy for stray spaces in the ARNs, including the `s3:ListBucket` line.
5. **The function times out**. Confirm the timeout is 15 minutes and memory is 2048 MB. The relay handles about 10,000 rows per second.

Relay Code

The same relay as the setup guide. Paste it into `lambda_function.py`, replacing the starter code, and leave the default handler, `lambda_function.lambda_handler`, as is.

```
"""Relay the AWS China CUR into a commercial-partition bucket, converted from CNY
to USD, keeping the Legacy CUR layout and manifests so CloudZero can process it
as native AWS billing data."""
import csv
import gzip
import io
import json
import os
import re
import re
import tempfile
import uuid
from datetime import date, datetime, timedelta, timezone
from decimal import Decimal, InvalidOperation

import boto3

CN_REGION = os.environ["CN_REGION"].strip()          # cn-north-1 or cn-northwest-1
CN_CUR_BUCKET = os.environ["CN_CUR_BUCKET"].strip()
CN_CUR_PREFIX = os.environ["CN_CUR_PREFIX"].strip()  # report path prefix in China
CUR_REPORT = os.environ["CUR_REPORT"].strip()        # report name, kept the same
CN_SECRET_ID = os.environ["CN_SECRET_ID"].strip()
DEST_BUCKET = os.environ["DEST_BUCKET"].strip()
DEST_PREFIX = os.environ["DEST_PREFIX"].strip()      # report path prefix in the destination
FX_RATES_KEY = os.environ["FX_RATES_KEY"].strip()
AUDIT_PREFIX = os.environ.get("AUDIT_PREFIX", "cur-relay-audit").strip()
LOOKBACK_DAYS = int(os.environ.get("LOOKBACK_DAYS", "10").strip())

# A column is converted when its category holds money and its name ends in one
# of these words, e.g. lineItem/UnblendedCost, pricing/publicOnDemandRate,
# savingsPlan/TotalCommitmentToDate, reservation/UpfrontValue.
MONEY_CATEGORIES = {"lineItem", "pricing", "reservation", "savingsPlan", "discount"}
MONEY_SUFFIX = re.compile(r"(Cost|Rate|Fee|Commitment|Discount|UpfrontValue)$")
CURRENCY_COLUMNS = {"lineItem/CurrencyCode", "pricing/currency"}
ONE = Decimal(1)

def dec(value):
    try:
        return Decimal(value) if value else Decimal(0)
    except InvalidOperation:
        return Decimal(0)

PLACES = Decimal("0.0000000001") # CUR money columns carry at most 10 decimal places

def scale(value, factor):
    try:
        return format((Decimal(value) * factor).quantize(PLACES).normalize(), "f")
    except InvalidOperation:
        return value # leave anything non-numeric untouched

def is_money(column):
    category, _, name = column.partition("/")
    return category in MONEY_CATEGORIES and bool(MONEY_SUFFIX.search(name))
```

```

def next_month(d):
    return (d.replace(day=28) + timedelta(days=4)).replace(day=1)

def billing_period_id(start):
    return f"{start:%Y%m%d}-{next_month(start):%Y%m%d}"

def periods_to_process(event):
    if event and event.get("periods"): # {"periods": ["2026-08"]} restates a month
        return [date.fromisoformat(p + "-01") for p in event["periods"]]
    today = datetime.now(timezone.utc).date()
    current = today.replace(day=1)
    periods = [current]
    if today.day <= LOOKBACK_DAYS: # AWS finalizes last month early in this one
        periods.insert(0, (current - timedelta(days=1)).replace(day=1))
    return periods

def china_s3():
    secret = json.loads(
        boto3.client("secretsmanager").get_secret_value(SecretId=CN_SECRET_ID)["SecretString"]
    )
    # A cn-* region makes boto3 use the amazonaws.com.cn endpoints
    return boto3.client(
        "s3",
        region_name=CN_REGION,
        aws_access_key_id=secret["access_key_id"],
        aws_secret_access_key=secret["secret_access_key"],
    )

def read_json(s3, bucket, key):
    try:
        return json.loads(s3.get_object(Bucket=bucket, Key=key)["Body"].read())
    except s3.exceptions.NoSuchKey:
        return None

def relay_file(cn, dest, src_key, dst_key, rate, totals):
    """Stream one CUR file, convert money columns to USD, upload it to the destination."""
    fd, path = tempfile.mkstemp(suffix=".csv.gz")
    os.close(fd)
    body = cn.get_object(Bucket=CN_CUR_BUCKET, Key=src_key)["Body"]
    with gzip.GzipFile(fileobj=body) as gin, \
        gzip.open(path, "wt", encoding="utf-8", newline="") as out:
        reader = csv.reader(io.TextIOWrapper(gin, encoding="utf-8", newline=""))
        writer = csv.writer(out, lineterminator="\n")
        header = next(reader)
        writer.writerow(header)
        money = [i for i, col in enumerate(header) if is_money(col)]
        currency = [i for i, col in enumerate(header) if col in CURRENCY_COLUMNS]
        code_i = header.index("lineItem/CurrencyCode")
        cost_i = header.index("lineItem/UnblendedCost")
        for row in reader:
            code = (row[code_i] or "CNY").upper()
            if code == "CNY":
                factor = rate
                totals["unblended_cny"] += dec(row[cost_i])
            elif code == "USD":
                factor = ONE

```

```

        else:
            raise ValueError(f"Unexpected currency {code} in {src_key}")
        for i in money:
            if row[i]:
                row[i] = scale(row[i], factor)
        for i in currency:
            if row[i]:
                row[i] = "USD"
        totals["unblended_usd"] += dec(row[cost_i])
        totals["rows"] += 1
        writer.writerow(row)
    dest.upload_file(path, DEST_BUCKET, dst_key)
    os.remove(path)
    return [header[i] for i in money]

def process_period(cn, dest, start, rates):
    period = billing_period_id(start)
    month = f"{start:%Y-%m}"
    if month not in rates:
        raise RuntimeError(f"No CNY to USD rate for {month} in {FX_RATES_KEY}")
    rate = Decimal(str(rates[month]))

    manifest_name = f"{CUR_REPORT}-Manifest.json"
    src_dir = f"{CN_CUR_PREFIX}/{CUR_REPORT}/{period}"
    dst_dir = f"{DEST_PREFIX}/{CUR_REPORT}/{period}"

    source = read_json(cn, CN_CUR_BUCKET, f"{src_dir}/{manifest_name}")
    if source is None:
        print(f"{period}: no China CUR yet, skipping")
        return

    # A new assembly ID whenever AWS publishes a new report version or the rate
    # changes, so CloudZero sees a new version to process. Otherwise, skip.
    assembly = str(uuid.uuid5(uuid.NAMESPACE_URL, f"{source['assemblyId']}|{rate}"))
    current = read_json(dest, DEST_BUCKET, f"{dst_dir}/{manifest_name}")
    if current and current.get("assemblyId") == assembly:
        print(f"{period}: already up to date")
        return

    totals = {"rows": 0, "unblended_cny": Decimal(0), "unblended_usd": Decimal(0)}
    dest_keys, converted = [], set()
    for src_key in source["reportKeys"]:
        dst_key = f"{dst_dir}/{assembly}/{src_key.rsplit('/', 1)[-1]}"
        converted.update(relay_file(cn, dest, src_key, dst_key, rate, totals))
        dest_keys.append(dst_key)

    # Same manifest AWS wrote, pointed at the new bucket, assembly and files
    manifest = {k: v for k, v in source.items() if k != "additionalArtifactKeys"}
    manifest.update({"assemblyId": assembly, "bucket": DEST_BUCKET, "reportKeys": dest_keys})
    body = json.dumps(manifest, indent=2)
    dest.put_object(Bucket=DEST_BUCKET, Key=f"{dst_dir}/{assembly}/{manifest_name}",
                    Body=body, ContentType="application/json")

    dest.put_object(
        Bucket=DEST_BUCKET,
        Key=f"{AUDIT_PREFIX}/{period}/{assembly}.json",
        Body=json.dumps({
            "period": period,
            "source_assembly_id": source["assemblyId"],
            "assembly_id": assembly,
            "rate_usd_per_cny": str(rate),
            "rows": totals["rows"],

```

```

        "unblended_cny": format(totals["unblended_cny"].quantize(Decimal("0.01")), "f"),
        "unblended_usd": format(totals["unblended_usd"].quantize(Decimal("0.01")), "f"),
        "converted_columns": sorted(converted),
    }, indent=2),
    ContentType="application/json",
)

# Written last: the top-level manifest is what points readers at the new version
dest.put_object(Bucket=DEST_BUCKET, Key=f"{dst_dir}/{manifest_name}",
                Body=body, ContentType="application/json")
print(f"{period}: {totals['rows']} rows, CNY {totals['unblended_cny']} "
      f"-> USD {totals['unblended_usd']} at {rate}")

def lambda_handler(event=None, context=None):
    cn = china_s3()
    dest = boto3.client("s3")
    rates = read_json(dest, DEST_BUCKET, FX_RATES_KEY)
    if rates is None:
        raise RuntimeError(f"Rates file {FX_RATES_KEY} not found in {DEST_BUCKET}")
    for start in periods_to_process(event):
        process_period(cn, dest, start, rates)

handler = lambda_handler # alias, if you save the file as relay.py and use relay.handler

```

● SUPPORT

Need a hand?

If you hit issues, reach out to your CloudZero contact. Include:

- The relay's log output and audit record from Step 6
- What you saw in CloudZero Explorer in Steps 8 and 9
- The connection name and bucket path



© CloudZero · Testing the AWS China Relay in a Sandbox · Last updated 2026-09-24